Austrian ICD-10 Educational Prototype

User Guide

Installation, learner workflow, maintenance, and troubleshooting

At a glance

This guide is for people who want to install and use the working thesis prototype. The quickest route uses Docker Compose and the project's published container images. No local PHP, MySQL, Node.js, Python, or Composer installation is required.

Guide revision	0.7
Last updated	9 August 2026
Prototype data	PROTOBASE-1.1 / BMASGPK 2026 (6 patients, 25 questions)
Repository	github.com/junomarx/bsc-thesis-icd10

Contents

1 Quick introduction

Austrian ICD-10 Educational Prototype is a bachelor-thesis software artefact for practising a small, explicitly defined subset of Austrian ICD-10 coding. It presents six synthetic teaching patients, each containing several independent coding questions; the learner answers one question at a time by selecting a supported code (or **None of the above**), and each answer is evaluated with deterministic rules. Feedback is returned as **Correct**, **Suboptimal**, or **Incorrect**, together with an explanation and, where applicable, a suggested improvement. The interface is available in English and German.

The prototype is deliberately small. It is intended to demonstrate and test the project's rule-based educational approach, not to provide a complete coding platform.

Educational use only

The patients and questions are synthetic. The prototype does not diagnose patients, provide clinical decision support, perform official coding, or support reporting or reimbursement. Do not enter real patient data.

2 Installation

2.1 What you need

Before starting, install and start the following:

- **Docker Desktop** on macOS or Windows, or Docker Engine on Linux;
- **Docker Compose v2**, available through the `docker compose` command;
- **Git** to download and update the repository;
- a modern web browser; and
- an internet connection for the initial download of the repository and container images.

Check Docker before continuing:

```
docker --version
docker compose version
```

Both commands should print version information. If Docker reports that it cannot connect to the daemon, open Docker Desktop or start the Docker service.

2.2 Install and start with the published images

Open a terminal and run:

```
git clone https://github.com/junomarx/bsc-thesis-icd10.git
cd bsc-thesis-icd10
docker compose pull
docker compose up -d --wait app
```

The first start downloads several images and may take a few minutes. Compose then starts MySQL, loads the versioned prototype baseline, and starts the web application in the correct order.

The project-owned images are published for both `linux/amd64` (Intel/AMD computers) and `linux/arm64` (including Apple Silicon). Docker selects the matching variant automatically.

If an older image tag has no Apple Silicon variant

If `docker compose pull` reports `no matching manifest for linux/arm64/v8`, the registry still has an older, AMD64-only publication. Use the native source-build procedure in Section 2.4; it works without emulation and does not depend on the registry containing the current multi-platform release.

Open `http://localhost:5860` in a browser.

Optionally verify the HTTP endpoint from a terminal:

```
curl http://127.0.0.1:5860/api/health
```

The expected response is:

```
{"status":"ok"}
```

What was installed?

Docker created containers for the database, one-time baseline loader, and application, plus a named database volume. The source checkout remains a normal folder and can be updated with Git. Learner attempts are not stored.

2.3 Use a different local port

Port 5860 is the default. If it is already in use, stop the stack and choose another port. On macOS or Linux:

```
docker compose down
APP_HTTP_PORT=8081 docker compose up -d --wait app
```

In Windows PowerShell:

```
docker compose down
$env:APP_HTTP_PORT = "8081"
docker compose up -d --wait app
```

Then open `http://localhost:8081`. The selected environment value remains in the current terminal session; clear or change it before a later start if you want to return to port 5860.

2.4 Build locally instead of pulling images

If a published image is unavailable for the host architecture, or if you want to run the exact source in the checkout, build the two images needed for normal use locally:

```
docker compose build bootstrap app
docker compose up -d --wait app
```

This takes longer on the first run because Docker builds the frontend, backend, and baseline-loader

images. It still does not require the language toolchains to be installed directly on the host. On Apple Silicon, the resulting `app` and `bootstrap` images are native ARM64 images.

3 Using the prototype

3.1 Switch language

An **EN | DE** switch sits in the top-right corner of every screen. The choice is remembered in the browser for later visits, and defaults to German or English automatically based on the browser's own language setting on first visit. Switching language translates all interface text, the six patient summaries, all 32 context records, all 25 question titles and prompts, displayed code designations, and feedback explanations. German mode uses the official Austrian source-catalogue designations; English mode uses reviewed British-English interface presentations for all 87 displayed codes because the repository has no official English counterpart. These presentations do not alter the underlying catalogue records. The application fails its localization checks if a required entry is missing, so it does not silently mix English and German at runtime.

3.2 Switch appearance

A small sun or moon button sits beside the language switch in the header. Use it to switch the whole application between light and dark appearance, including the tutorial and feedback views. The compact button shows its action when hovered and provides a localized accessible toggle state; it remains available on every screen.

On the first visit, the prototype follows the browser or operating system's light/dark preference. After the button is used, that explicit choice is remembered in this browser and reused on later visits. The appearance value is stored only as a local interface preference; it is never sent to the server and does not contain learner answers or patient data.

3.3 How this works: interactive tutorial

On the first visit in a browser, a four-step **How this works** tutorial opens over the application. Use **Next** and **Back** to move through choosing a patient, reviewing the patient record, answering one question at a time, and interpreting feedback. The final step explains the three feedback classes with the same icon and colour used later in the actual results. Choose **Skip tutorial**, the close button, or press Escape to leave it at any time. You can also click or tap the dimmed area outside the tutorial card; a click inside the card does not close it.

After the tutorial is closed, the browser remembers that it has been seen and does not open it automatically on later page loads. It remains available manually from the persistent **How this works** button in the page header on every screen. Clearing this site's browser storage makes the next load a first visit again. This small browser preference is never sent to the server and does not contain learner answers or patient data.

3.4 Choose a patient

On the roster, the six synthetic patients appear as a grid of equally sized cards. Each card shows the patient's display name, age, sex, a neutral complexity label (*Foundational* or *Involved*), and its question count (3, 5, or 6 depending on the patient), followed by a one-sentence summary of that patient's record. Difficulty is informational only — no patient is locked, and any patient may be selected in any order.

A patient already fully answered in this browser session shows a *Completed* badge on its card, and a one-line summary above the grid tracks overall progress ("n of 6 patients completed").

This record lives only for the current browser session (cleared when the browser tab/session ends, not just on page reload) — nothing is sent to or stored by the server. A **Reset progress** link appears next to the summary whenever at least one patient has been completed, and clears every completion mark after a confirmation prompt. Select a patient to begin.

3.5 Review the patient record

Choosing a patient opens its first question directly. A **Show patient info** button reveals a collapsible panel with the patient's identity, a general summary, and its documented context items (conditions, history, current findings), each labelled by information type — for example, whether a detail comes from a documented record versus a current examination finding, which matters for questions about a patient who cannot provide their own history. This panel can be opened and closed at any point without losing the current question's state, and remains reachable throughout the patient's questions via the same button.

3.6 Answer a question

Each question shows its position ("Question 2 of 5"), a matching visual progress bar, the patient panel, the question prompt, and a list of answer options: several supported ICD-10 codes with their designation, plus a dedicated **None of the above** option. Select one option, then choose **Submit answer**; the button remains disabled until an option is selected. Question order is shuffled independently each time a patient is started or replayed, so the same patient may present its questions in a different sequence on a later attempt — this never changes which questions or options exist, only their order.

Only the options shown are offered; free-text or arbitrary code submission is not part of the learner interface. An **Exit to patient list** link is available at any point during a question, in case you want to return to the patient list before finishing — a confirmation is required, since the current, unfinished attempt on that patient is not saved.

3.7 Interpret the feedback

Each result is shown with a matching icon and colour alongside its text label, so the outcome does not depend on colour perception alone.

Result	Meaning in this prototype
Correct	The documented information supports the submitted code as an appropriate response for the question.
Suboptimal	The documented information supports the answer, but also supports a more specific code. A suggested improvement is shown.
Incorrect	A hard rule was violated, for example a status restriction, required coding depth, conflict with a documented fact, or a wrong temporal/status context — or, for **None of the above**, a declared acceptable code was in fact among the displayed options.
Not evaluated	The request could not be classified because a required question or code relationship was unavailable. This is normally a data or application problem, not an expected learner outcome.

The explanation beneath the result states the rule-relevant reason, in the selected interface language. A collapsed **Technical details** section is available beneath it, showing the determining rule and criterion for anyone who wants to inspect exactly which rule fired — useful for demonstration, not required reading for normal use. Once submitted, an answer is locked for the rest of that attempt; choose **Next question**, or **Review patient** after the last question of that patient.

3.8 Review a completed patient

After the last question, a review screen shows the patient's name with a completion acknowledgement, raw counts of correct/suboptimal/incorrect results (never a weighted score), and a list of every question with its result. From here, choose **Play again** to reattempt the same patient with its questions reshuffled, or **Choose another patient** to return to the patient list.

4 Starting, stopping, and updating

Run all commands in the repository folder containing `docker-compose.yml`.

4.1 Check status and logs

```
docker compose ps
docker compose logs app
docker compose logs db
docker compose logs bootstrap
```

Add `-f` to a log command to follow new output, for example `docker compose logs -f app`. Press Ctrl+C to stop following the log; this does not stop the container.

4.2 Stop and restart

Stop the containers while keeping the database volume:

```
docker compose down
```

Start the prototype again later:

```
docker compose up -d --wait app
```

4.3 Update to the newest repository and images

```
git pull --ff-only
docker compose pull
docker compose up -d --wait app
```

The bootstrap service checks the versioned baseline during startup and is safe to run again when the baseline is unchanged.

4.4 Reset the prototype data

This removes the database volume

The following command deletes the local Compose database volume. In the current prototype that volume contains only the reproducible, versioned baseline—not learner accounts or attempt histories. Do not reuse this command unchanged if persistent user data is added in a future version.

```
docker compose down -v
```

```
docker compose up -d --wait app
```

Resetting is useful after an incompatible `mysql:latest` upgrade or when you need a clean copy of the baseline.

5 Optional: run the automated tests

The committed PHPUnit suite (`app/tests/`) targets the current six-patient/25-question model described in this guide: 83 unit tests, 163 integration tests (against a live MySQL baseline), 12 browser-driven end-to-end tests, and 4 frontend localization-contract tests, all passing as of the last verified run (see `docs/CHANGELOG.md` for exact counts and how they were verified).

The application does not start test tooling during normal use. To run the containerized unit, integration, and browser-driven test suite:

```
docker compose --profile test up -d --wait selenium
docker compose --profile test run --rm test
```

If the test image also needs to be built from the checkout, run this once before the commands above:

```
docker compose --profile test build test
```

A successful run ends with PHPUnit reporting no failures or errors. The browser tests use a separate Selenium container and may take longer on the first run.

Stop the application and test services when finished:

```
docker compose --profile test down
```

The database volume is retained unless `-v` is added.

6 Troubleshooting

6.1 Docker cannot connect

If a command reports that the Docker daemon is unavailable, start Docker Desktop or the Linux Docker service and retry `docker compose ps`.

6.2 Port 5860 is already allocated

Use the different-port procedure in Section 2.3. If a previous copy of this project is still running, `docker compose down` from its repository folder may release the port.

6.3 The database is unhealthy after an update

The project deliberately uses `mysql:latest`. MySQL may reject an existing data volume after a major-version jump. Inspect the database log:

```
docker compose logs db
```

If the log reports an unsupported in-place upgrade, use the reset procedure in Section 4.4. This deletes and reconstructs the local baseline database.

6.4 The page does not load or patients do not appear

Check the service state and health endpoint:

```
docker compose ps
curl http://127.0.0.1:5860/api/health
docker compose logs app
docker compose logs bootstrap
```

If you selected a different port, replace 5860. A failed bootstrap prevents the application from starting because the baseline is a required dependency; the bootstrap log contains the cause.

6.5 Pull reports “no matching manifest”

This error means the selected registry tag does not contain an image for the computer’s architecture. It was observed on Apple Silicon with the project’s original AMD64-only publication. The publishing workflow now produces both AMD64 and ARM64 variants. Update the checkout and retry the pull if a custom, cached, or historical tag still exposes the earlier single-architecture index.

Run the native fallback from Section 2.4:

```
docker compose build bootstrap app
docker compose up -d --wait app
```

Do not add a permanent `platform: linux/amd64` override merely to hide the error; that would force slower emulation on Apple Silicon after native images are available.

6.6 The browser tests cannot start

Confirm that Docker has enough memory available and that ports 4444 and 7900 are not already occupied. The normal learner-facing application does not need these ports; they belong only to the optional Selenium service.

7 Data, privacy, and scope

- All bundled patients and questions are synthetic teaching content.
- The current interface has no account system and does not persist learner submissions or attempt history anywhere server-side. The per-patient *Completed* marks described in Section 3 live only in the browser, only for the current session, and are never sent to the server.
- The selected language and appearance, and the fact that the first-visit tutorial has been dismissed, are stored locally in the browser so those three interface preferences survive a reload. None contains a learner response or is sent to the server.
- The database volume holds the versioned prototype catalogue, patient, and question data, including the rule-support facts each question is evaluated against. The independent reference-response oracle is retained exclusively as test-harness data. It is not imported into the runtime database and is unavailable to the production application path.

- The implemented catalogue is a small, traceable subset of the Austrian ICD-10 BMASGPK 2026 material, not the entire catalogue.
- Results express the prototype's declared rules and question relationships; they are not medical or official coding advice.

8 Further project documentation

This guide covers installation and day-to-day use. The repository contains the following deeper references:

- `docs/IMPLEMENTATION_SPECIFICATION.md` — exact implemented schema, API, build, and deployment contracts;
- `docs/DEVELOPMENT_DOCUMENTATION.md` — technology and design rationale;
- `docs/REQUIREMENTS_TRACEABILITY.md` — requirement-to-evidence mapping;
- `docs/CHANGELOG.md` — dated implementation and documentation history; and
- `HANDOFF.md` — current project snapshot and remaining work.

The editable source for this document is `docs/USER_GUIDE.tex`; the compiled edition is `docs/USER_GUIDE.pdf`. Both are maintained in the repository so the readable PDF and its source can be reviewed together.

A Command summary

Command	Purpose
`docker compose pull`	Download published images.
`docker compose up -d -wait app`	Start the application and its dependencies.
`docker compose ps`	Show service state.
`docker compose logs app`	Show application logs.
`docker compose down`	Stop containers; keep the database volume.
`docker compose down -v`	Stop containers and delete the database volume.
`docker compose build bootstrap app`	Build the normal-use images natively from the local checkout.
`docker compose -profile test run -rm test`	Run the full automated test suite after Selenium is started.